



CSCI-SHU 360: MACHINE LEARNING

FINAL COMPETITION REPORT - FALL 2025

Urban Sound Classification with Multi-Model Ensemble Learning

Sam Shi
ss19608@nyu.edu

1 Introduction

Urban sound classification aims to identify environmental audio events such as car horns, sirens, dog barks, and gunshots from short audio recordings. Unlike speech or music recognition, urban sound data often contains transient, noisy, and highly overlapping acoustic patterns, making the task particularly challenging. In this competition, we are given a dataset of 4-second audio clips labeled into 10 urban sound categories, and the goal is to maximize classification accuracy on a held-out test set evaluated via Kaggle. The dataset follows an 8-fold structure, which allows for robust cross-validation while preventing data leakage between training and validation samples. Classification accuracy is used as the primary evaluation metric. My work begins with a standard CNN baseline and evolves into a large-scale experimental pipeline that integrates model comparison, data augmentation, ensemble learning, and systematic hyperparameter tuning.

2 Data Preprocessing

This section describes, step by step, how I transform each raw audio clip into the final tensor input used by my models. All preprocessing is implemented from scratch in the dataset layer (i.e., inside `data/dataset.py`), so the training loop remains model-agnostic and reproducible across folds, seeds, and architectures.

2.1 Step 1: Load waveform and standardize length

Each audio file is loaded as a mono waveform using `librosa.load`. I fix the sample rate to 22,050 Hz for all experiments. Since clips may be shorter than the target length, I enforce a strict fixed-length representation:

- Target duration: 4.0 seconds
- Target length in samples: $L = 22050 \times 4 = 88200$

If the waveform is longer than L , it is truncated. If it is shorter than L , it is zero-padded at the end. Finally, I normalize waveform amplitude using `librosa.util.normalize` so that each clip has a comparable scale before feature extraction. In code, this behavior is implemented in `_load_wave(path, sr, duration)`.

2.2 Step 2: Convert waveform to time-frequency representation

Instead of feeding raw waveform directly, I convert each clip into a log-scaled Mel spectrogram. This produces an image-like time-frequency feature that is standard in audio classification and works well with CNN/CRNN models.

For a given waveform $y(t)$, I compute:

- FFT window size: n_fft
- Hop length: hop
- Number of Mel bins: $n_mels = 128$

The Mel spectrogram is computed by:

$$S_{\text{mel}} = \text{MelSpec}(y; sr, n_fft, hop, n_mels)$$

Then I convert power spectrogram to log scale (decibel scale):

$$S_{\text{log}} = 10 \log_{10}(S_{\text{mel}} + \epsilon)$$

In practice, this step is implemented using `librosa.feature.melspectrogram` and `librosa.power_to_db`.

2.3 Step 3: Normalize the spectrogram to a stable numeric range

After computing log-Mel spectrograms, I apply min-max normalization to map values into $[0, 1]$:

$$\hat{S} = \frac{S - \min(S)}{\max(S) - \min(S) + 10^{-8}}$$

This ensures the input range is stable across clips and improves optimization stability during training. This is implemented in a helper function `_norm01(m)`.

2.4 Step 4A: ResNet feature pipeline (Mel + Delta + Delta2)

For ResNet-based models, the goal is to provide CNN-friendly multi-channel inputs that capture both static spectral patterns and local temporal dynamics. After obtaining the normalized base Mel spectrogram $\hat{S}$, I compute the first-order and second-order temporal derivatives:

- $\Delta(\hat{S})$ (delta feature)
- $\Delta^2(\hat{S})$ (delta-delta feature)

These are computed using `librosa.feature.delta`. The final ResNet input is a 3-channel tensor:

$$X_{\text{resnet}} = [\hat{S}, \Delta(\hat{S}), \Delta^2(\hat{S})]$$

so that the input shape becomes:

$$(3, 128, T)$$

This design is motivated by the fact that delta features often help capture short transitions and modulations that are important in environmental sounds, while still allowing the CNN to learn spatial patterns efficiently.

2.5 Step 4B: CRNN feature pipeline (Multi-resolution Mel stack)

For CRNN, the goal is to provide richer temporal-frequency resolution since CRNN explicitly models time evolution. Instead of delta features, I build a 3-channel **multi-resolution** log-Mel representation by varying n_fft :

$$n_fft \in \{400, 1024, 2048\}$$

Smaller n_fft provides better time resolution (useful for transient events), while larger n_fft provides better frequency resolution (useful for tonal or harmonic patterns). For each n_fft , I compute a normalized log-Mel spectrogram $\hat{S}_{n_fft}$, and stack them:

$$X_{\text{crnn}} = [\hat{S}_{400}, \hat{S}_{1024}, \hat{S}_{2048}]$$

The resulting input shape is also:

$$(3, 128, T)$$

but the three channels now represent different time-frequency tradeoffs rather than derivatives. This is particularly useful for CRNN because the recurrent module can exploit the temporal progression encoded differently by each resolution.

2.6 Step 5: Time cropping strategy (train vs. validation)

Even though each clip is standardized to 4 seconds, I do not always feed all 4 seconds into the model. Instead, I crop to a 3-second region. This reduces computation, adds augmentation diversity, and forces robustness to temporal positioning of events.

Training cropping: During training, I apply random cropping in time. For ResNet, this is implemented approximately by selecting a random segment length proportional to `crop_duration/duration`. For CRNN, I compute the crop length in frames using:

$$\text{frames_per_sec} = \frac{sr}{hop}, \quad T_{\text{crop}} = \lfloor \text{frames_per_sec} \times 3.0 \rfloor$$

and then choose a random starting frame.

Validation cropping: For validation, I use deterministic center cropping so evaluation is stable and comparable across epochs and seeds.

2.7 Step 6: On-the-fly data augmentation inside Dataset

All augmentations are applied on-the-fly in the dataset layer to avoid storing multiple copies of features and to ensure that each epoch sees different variations. Importantly, augmentation behavior is model-dependent.

ResNet augmentations: For ResNet experiments, I apply:

- Random time crop (3s segment)
- Additive noise (Gaussian, fixed small scale)
- Spectrogram mixing (**specmix**): replace a random time region with a time-shifted version of itself
- SpecAugment (time mask + frequency mask)

The SpecAugment strength is controlled by hyperparameters `TIME_MASK` and `FREQ_MASK`.

CRNN augmentations: For CRNN experiments, I apply:

- Random time crop (3s segment)
- Additive noise with probability p and random σ range
- Probabilistic SpecAugment with multiple time masks and frequency masks

The augmentation is controlled by:

$$p_{\text{noise}}, \sigma_{\text{min}}, \sigma_{\text{max}}, p_{\text{specaug}}, \text{mask sizes and counts}$$

This design is motivated by the fact that CRNN is more sensitive to temporal corruption, so augmentation must be strong enough to regularize but not so strong that it destroys sequential structure.

2.8 Step 7: Convert features to tensors and return standardized outputs

After feature extraction and augmentation, each sample is converted into PyTorch tensors:

$$x \in \mathbb{R}^{3 \times 128 \times T}, \quad y \in \{0, \dots, 9\}$$

The dataset returns a tuple `(x, y, sid)` where `sid` is the sample ID when available. This standard output interface allows the same training runner to work across all model families.

2.9 Summary

In summary, my preprocessing pipeline enforces a consistent waveform standardization step, then creates model-specific spectrogram representations:

- **ResNet:** log-Mel + delta + delta2 (3 channels), emphasizing spectral patterns and local transitions.
- **CRNN:** multi-resolution log-Mel stack (3 channels from different FFT sizes), emphasizing complementary temporal-frequency resolutions for sequential modeling.

All augmentation and cropping are integrated directly into the dataset so that experiments are fully reproducible and modular.

2.10 Configurable Data Preprocessing Across Experiments

This subsection describes a key design choice in my pipeline: data preprocessing is jointly optimized with model architecture and training hyperparameters.

An important aspect of my pipeline is that data preprocessing is not fixed to a single deterministic configuration. Instead, preprocessing behavior varies across different experimental runs through changes in random seeds and hyperparameters. Each experiment configuration is generated from a base setup and then perturbed along multiple sensitive dimensions, including learning rate, label smoothing, EMA timing, and data augmentation strength. As a result, even when using the same raw audio clip, different experiments may apply different random crops, different noise realizations, different SpecAugment masks, and different mixup schedules.

Concretely, the random seed controls all stochastic components in preprocessing, such as time cropping positions, noise injection, and masking locations. In addition, hyperparameters such as the number and size of time masks, frequency masks, noise probability, and mixup activation epoch are also part of the experiment configuration. This means that each (model, seed, fold) combination effectively observes a slightly different version of the training data distribution. Rather than treating randomness as noise, I explicitly exploit it as a form of controlled data diversification.

This design turns data preprocessing itself into a tunable component of the learning system. By training multiple models under different preprocessing regimes and then ensembling them, the final prediction benefits from exposure to a broader range of augmented data views. In practice, this significantly reduces overfitting and improves robustness, especially for transient or ambiguous sound events. Therefore, in my framework, preprocessing is not a static preprocessing step performed once, but an integral part of the multi-seed, multi-experiment learning strategy.

3 Methodology

This section describes the design of my model architectures and the overall training strategy. Rather than relying on a single model or a single configuration, my methodology is built around a unified multi-model, multi-seed training pipeline. Architectural diversity, stochastic training behavior, and ensemble learning are jointly leveraged to improve robustness and final performance.

3.1 Model Design Overview

I explored two main types of models for urban sound classification: convolutional neural networks (ResNet variants) and a Convolutional Recurrent Neural Network (CRNN). These architectures were selected to capture complementary characteristics of audio signals.

ResNet-based models treat log-Mel spectrograms as image-like inputs and are effective at learning discriminative spectral patterns through deep convolutional hierarchies. They perform particularly well on short, impulsive sound events such as *gun shot*, *dog bark*, and *car horn*, where

localized, high-energy spectral patterns are most informative. I experimented with ResNet34, ResNet50, and later ResNet18. Through systematic validation experiments, I found that deeper models such as ResNet50 were slower to train and more prone to overfitting on this relatively small dataset, while ResNet18 provided a better balance between model capacity, training stability, and generalization.

To complement the ResNet models, I designed a CRNN from scratch. The CRNN first applies convolutional layers to extract local time–frequency features, and then feeds the resulting feature sequence into a bidirectional GRU. This recurrent component enables explicit modeling of temporal dependencies across frames. As a result, the CRNN performs especially well on temporally structured or sustained sound events such as *siren*, *engine idling*, *children playing*, and *street music*, where long-term temporal evolution is critical for correct classification.

The final CRNN architecture consists of a CNN front-end, a two-layer bidirectional GRU, and an attention-based temporal pooling mechanism. The attention module allows the model to focus on the most informative time segments within each audio clip, further improving performance on non-uniform and evolving sound events. Figure 1 provides a high-level overview of the complete training, inference, and ensemble workflow, highlighting the modular design and the interactions between configuration, data processing, model training, and prediction aggregation.

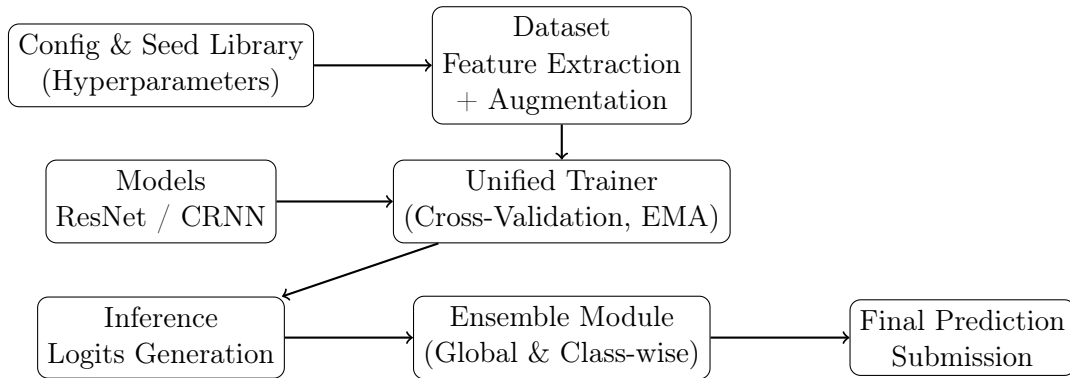


Figure 1: Overall structure of the proposed training, inference, and ensemble pipeline.

3.2 Unified Training Pipeline

All models are trained through a unified training interface implemented in a shared runner module. This design allows different architectures to share the same training logic, data loading process, and evaluation protocol, ensuring fair and consistent comparison across models.

Training is performed using an 8-fold cross-validation strategy. For each fold, the model is trained on seven folds and validated on the remaining fold. Each training run is defined by a tuple of (model type, fold index, random seed, hyperparameter configuration) and is treated as an independent experiment.

3.3 Optimization and Regularization

All models are optimized using AdamW with cosine annealing learning rate scheduling. Weight decay and label smoothing are applied to regularize training and prevent over-confident predictions.

I further incorporate an Exponential Moving Average (EMA) of model parameters. After an initial warm-up period, validation is performed using the EMA-smoothed weights rather than the raw parameters, which consistently improves validation stability and generalization.

Mixup is optionally enabled depending on the experiment configuration. I found mixup to be particularly effective for ResNet-based models, while its impact on CRNN was more sensitive and

required careful tuning to avoid disrupting temporal structure.

3.4 Multi-Seed and Ensemble-Oriented Design

Instead of relying on a single random initialization, I explicitly train multiple variants of each model using different random seeds and slightly perturbed hyperparameters. These variations explore different optimization trajectories and increase model diversity.

All trained models output fold-specific and seed-specific logits, which are saved for later analysis. These logits form a clean interface between training and inference, enabling ensemble strategies without retraining models. In the final stage, predictions are combined using both global-weighted and class-wise weighted ensemble methods, with weights derived from out-of-fold validation performance.

Overall, this methodology integrates architectural diversity, robust optimization, and ensemble learning into a single coherent framework, enabling consistent and interpretable improvements over baseline models.

4 Experiments and Hyperparameter Tuning

In this section, I describe how hyperparameters were tuned during training and validation, what experimental techniques were explored, and the key observations that guided the evolution of the final pipeline. Rather than relying on a single configuration, the overall approach was highly iterative: I started from a strong but limited baseline, identified bottlenecks through validation performance and confusion matrix analysis, and gradually refined both model architecture and training strategy.

4.1 Baseline Configuration and Early Hyperparameter Exploration

The initial baseline employed an 8-fold cross-validation setup with a ResNet34 architecture. Using a standard configuration (learning rate 3×10^{-4} , batch size 32, and minimal data augmentation), validation accuracy stabilized at approximately 0.83. To better understand the sensitivity of the model to optimization choices, I systematically explored several core hyperparameters:

- **Learning rate:** Values ranging from 1×10^{-4} to 5×10^{-4} were evaluated. Higher learning rates accelerated convergence but led to unstable validation behavior, whereas lower learning rates improved stability at the cost of slower training.
- **Batch size:** Smaller batch sizes (24–28) yielded marginal accuracy gains but significantly increased training time. A batch size of 32 was selected as a practical compromise.
- **Weight decay:** Increasing weight decay from 1×10^{-4} to 2×10^{-4} effectively reduced overfitting once stronger augmentation was introduced.

These early experiments established a stable optimization regime and provided a reliable reference point for subsequent architectural and preprocessing improvements.

4.2 Model Comparison Summary

To provide a concise overview of the major model configurations explored throughout this project, Table 1 summarizes the corresponding validation accuracy achieved as the pipeline evolved.

The results illustrate a clear progression in performance, highlighting the impact of stronger regularization, architecture selection, and ensemble learning on model robustness and generalization.

Table 1: Comparison of major model configurations and validation accuracy

Model	Augmentation Strength	Ensemble	Validation Accuracy
ResNet34	Weak	No	0.83
ResNet18	Strong	No	0.87
CRNN	Tuned (Temporal-aware)	No	0.86
ResNet18 + CRNN	–	Yes (Class-wise)	0.88

4.3 Progressive Data Augmentation Tuning

After establishing a stable baseline, diminishing returns from architectural changes alone became evident. Consequently, attention shifted toward data preprocessing and augmentation, which proved to be one of the most influential factors in improving performance.

Starting from a minimal augmentation setup, techniques were incrementally introduced:

- **Random time cropping:** Improved robustness to temporal variation and yielded consistent accuracy gains.
- **Additive noise:** Gaussian noise injection enhanced performance on noisy classes and reduced overfitting.
- **SpecAugment:** Time and frequency masking produced substantial improvements, though overly aggressive time masking negatively affected short transient events.
- **Mixup:** Although mixup slowed early convergence, it reduced variance across folds and improved final validation accuracy when sufficient training epochs were used.

A key observation was that augmentation strength must be matched to model capacity. Smaller models such as ResNet18 benefited from stronger augmentation, while more complex models (e.g., CRNN) required more conservative augmentation to preserve temporal structure.

4.4 Architecture Comparison and Model-Specific Tuning

Motivated by class-level performance differences observed in validation results, I compared ResNet18, ResNet34, and ResNet50 architectures. Despite its higher representational capacity, ResNet50 trained significantly slower and exhibited clear overfitting on this dataset. Even with early stopping and reduced training duration, it failed to outperform ResNet34.

In contrast, ResNet18 achieved improved generalization, reaching approximately 0.87 validation accuracy under similar training settings. This suggests that, for the given dataset size, a smaller model combined with stronger regularization and augmentation can outperform deeper architectures.

To address complementary error patterns revealed by confusion matrices, a CRNN model was introduced. ResNet-based models performed particularly well on short, impulsive sound events such as *gun shot*, *dog bark*, and *car horn*, which are characterized by localized, high-energy spectral patterns. In contrast, the CRNN model demonstrated clear advantages on temporally structured and sustained sound classes, including *siren*, *engine idling*, *children playing*, and *street music*, where explicit modeling of temporal dependencies is critical.

CRNN-specific tuning therefore focused on preserving temporal information rather than introducing aggressive local perturbations. This involved adjusting FFT sizes for multi-resolution Mel extraction, carefully controlling time-mask parameters, and moderating noise injection probability. Although the CRNN’s overall accuracy remained slightly below that of ResNet-based models, it consistently reduced confusion among temporally continuous sound classes.

4.5 Confusion Matrix Analysis and Ensemble Strategy

To better understand model-specific error patterns, normalized confusion matrices for the ResNet model, the CRNN model, and the final ensemble are shown in Figures 2–4.

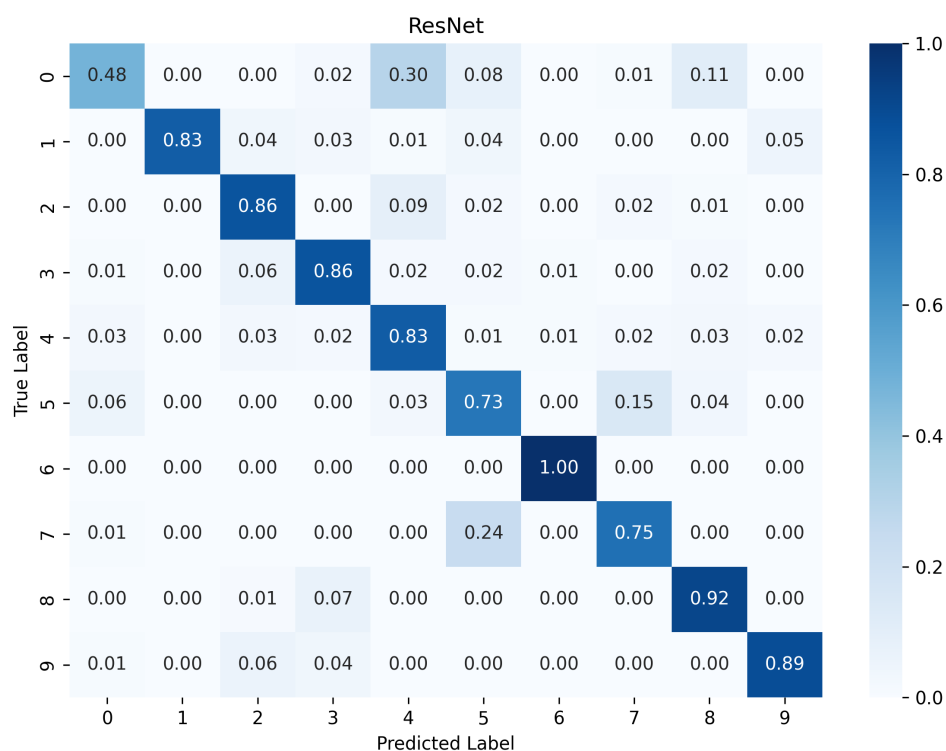


Figure 2: Normalized confusion matrix of one of the ResNet-based model.

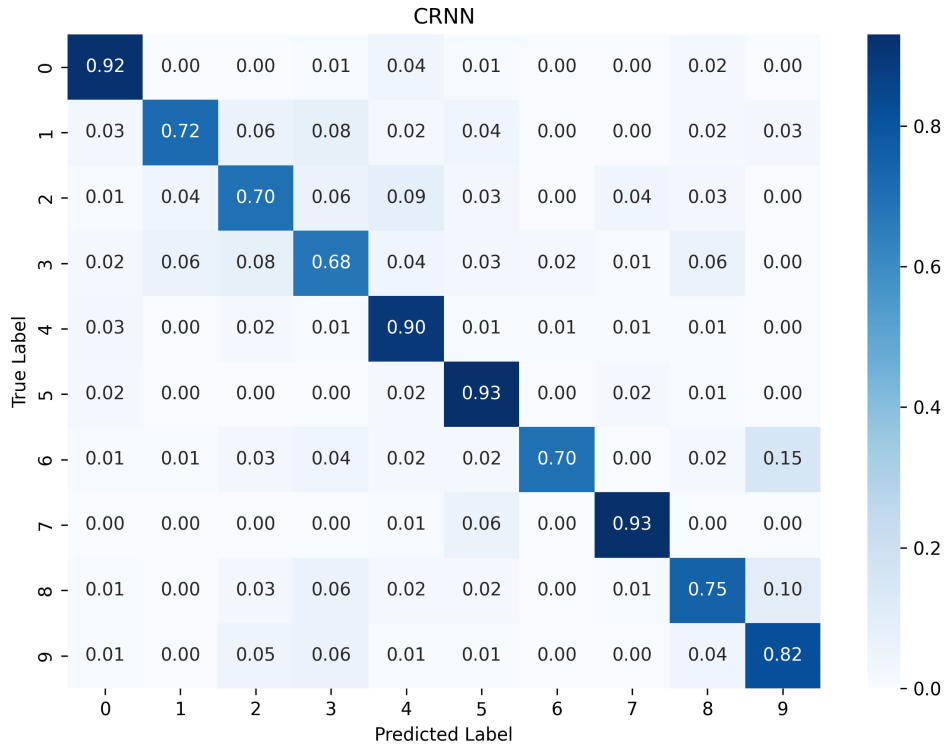


Figure 3: Normalized confusion matrix of one of the CRNN model.

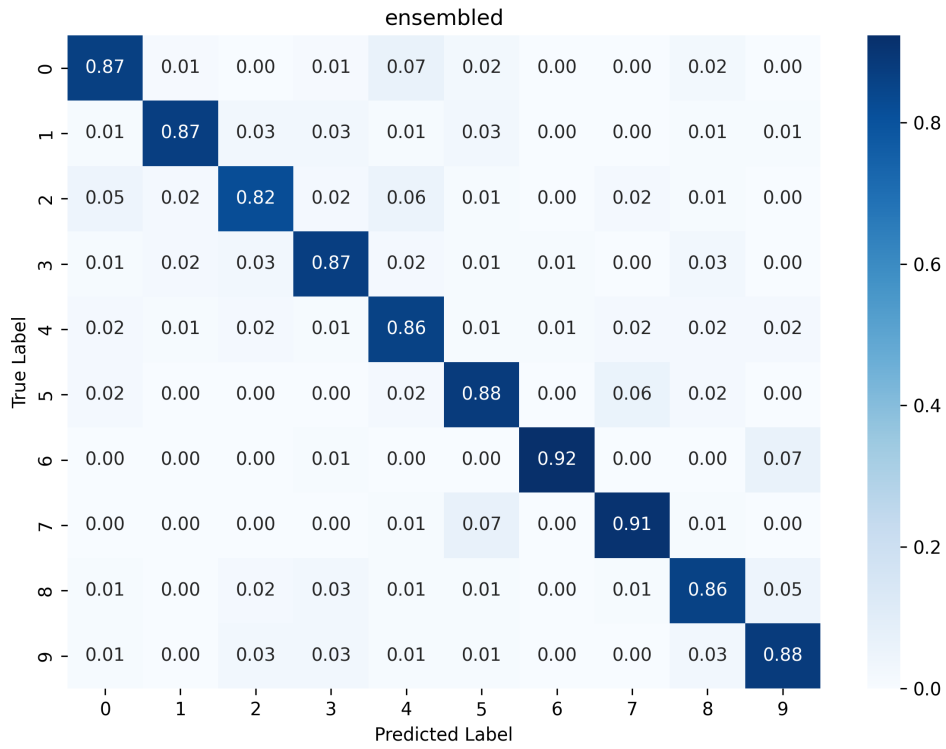


Figure 4: Normalized confusion matrix of the final class-wise ensemble model.

Figure 2 shows that the ResNet model performs particularly well on sound classes with strong and distinctive spectral characteristics. In particular, high accuracy is observed for short, impulsive events such as *gun shot* (class 6) and *dog bark* (class 3), as well as for sounds with clear harmonic or modulation patterns such as *siren* (class 8) and *street music* (class 9). These classes exhibit concentrated diagonal mass, indicating that their discriminative cues can be effectively captured by convolutional filters operating on local time–frequency patterns.

However, the ResNet model exhibits notable confusion among mechanically generated and low-frequency continuous sounds. For example, *air conditioner* (class 0) is frequently misclassified as *drilling* (class 4) or *engine idling* (class 5), while *jackhammer* (class 7) and *engine idling* (class 5) are also commonly confused with each other. These classes share similar broadband or quasi-stationary spectral structures, making them difficult to distinguish based solely on static spectral features.

In contrast, Figure 3 demonstrates that the CRNN model consistently reduces confusion among temporally structured and sustained sound classes. By explicitly modeling long-range temporal dependencies, the CRNN better distinguishes sounds such as *engine idling* (class 5), *jackhammer* (class 7), and *air conditioner* (class 0), where temporal consistency and modulation patterns provide important discriminative information beyond local spectral content. Improvements are also observed for classes such as *children playing* (class 2) and *street music* (class 9), which involve evolving temporal structures.

Motivated by these complementary error patterns, I constructed a class-wise weighted ensemble that selectively emphasizes the model with stronger per-class performance. As shown in Figure 4, the ensemble confusion matrix exhibits a clearer diagonal structure across nearly all classes, with reduced off-diagonal mass compared to either individual model. This indicates that the ensemble effectively combines the spectral discrimination ability of ResNet with the temporal modeling capacity of CRNN, resulting in a more balanced and robust classifier overall.

Concretely, ensemble inference was performed at the logit level using a class-conditional expert selection strategy. During validation, model variants were evaluated on a per-class basis, and for each sound class a small subset of top-performing models was identified. At inference time, all candidate models produced logits for a given input sample. For each class, only logits from the corresponding expert models were aggregated, while non-expert models for that class were excluded. The final prediction was obtained by concatenating the aggregated class-wise logits and applying a softmax operation.

4.6 Multi-Seed and Multi-Configuration Experiments

The ensemble strategies discussed in the previous section rely critically on the availability of diverse yet well-performing base learners. To construct such a pool of complementary models, I expanded the pipeline beyond simple multi-seed training to include controlled multi-configuration experiments, which serve as the foundation for the final ensemble.

Rather than performing an exhaustive grid search, each experiment variant was generated through small, carefully constrained deviations from a human-designed base configuration. This design ensures training stability while inducing sufficient diversity in optimization trajectories and regularization behavior, which is essential for effective ensemble learning.

For each model architecture, multiple independent variants were constructed by jointly perturbing several sensitive dimensions. These included offsetting random seeds to induce distinct optimization trajectories, applying small log-scale adjustments to the learning rate, and slightly varying label smoothing strength. In addition, the activation timing of EMA and mixup was adjusted within a narrow window to explore their interaction with early training dynamics.

For the CRNN model in particular, data augmentation itself was treated as part of the search space. In addition to optimizer-related perturbations, augmentation parameters such as time mask width, number of time masks, and SpecAugment application probability were varied across variants. This design reflects the higher sensitivity of sequence-based models to temporal cor-

ruption and allows the CRNN to observe a broader yet controlled range of augmented data distributions.

Each configuration was treated as an independent learner and trained using the same cross-validation protocol. Predictions from all variants were then aggregated through ensemble averaging and class-wise weighting. This multi-seed, multi-configuration strategy substantially reduced performance variance across folds and contributed to the final ensemble accuracy of approximately 0.88.

4.7 Key Findings

Overall, the experimental results reveal several important insights:

- **Model capacity must align with dataset scale and signal characteristics.** Deeper architectures such as ResNet50 did not yield additional benefits on this dataset and were more prone to overfitting, while smaller models with appropriate regularization achieved stronger generalization.
- **Data preprocessing and augmentation are first-class design choices.** Treating preprocessing and augmentation as tunable components, rather than fixed transformations, proved critical for improving robustness, particularly when combined with multi-seed training.
- **Architectural inductive biases lead to complementary error patterns.** Convolutional models excel at capturing localized spectral cues in impulsive events, whereas sequence-based models provide advantages for temporally structured and sustained sounds. These differences are clearly reflected in confusion matrix analysis.
- **Effective ensemble learning benefits from structured diversity.** Controlled perturbations of optimization, regularization, and augmentation parameters produced more reliable ensemble gains than relying on a single configuration or random restarts alone.
- **Class-wise ensemble weighting can exploit fine-grained model strengths.** Leveraging per-class performance differences resulted in a more balanced classifier with reduced class-specific confusion compared to global averaging.

These findings guided the design of my final pipeline and motivated the structured, modular framework used in this project.

5 Kaggle Score Screenshot

Figure 5 shows the leaderboard ranking!



Figure 5: Kaggle leaderboard ranking

Acknowledgements

I would like to sincerely thank Prof. Guo for the guidance and support provided throughout the semester.

I would also like to thank Tony for the insightful discussions and help with model analysis and hyperparameter tuning, especially during the later stages of the project. His feedback was invaluable in refining the final pipeline.

In addition, I would like to thank you for taking the time to review this report. Although the report is much longer than typical (I am not sure how much "detail" is the detail required), it reflects the significant amount of time and effort I invested into this project, and many components were difficult for me to omit. This project represents an important milestone in my college experience. Finally, I would like to thank myself for the persistence and dedication that allowed me to complete this work at the end of the semester.